

The Alan Turing Institute

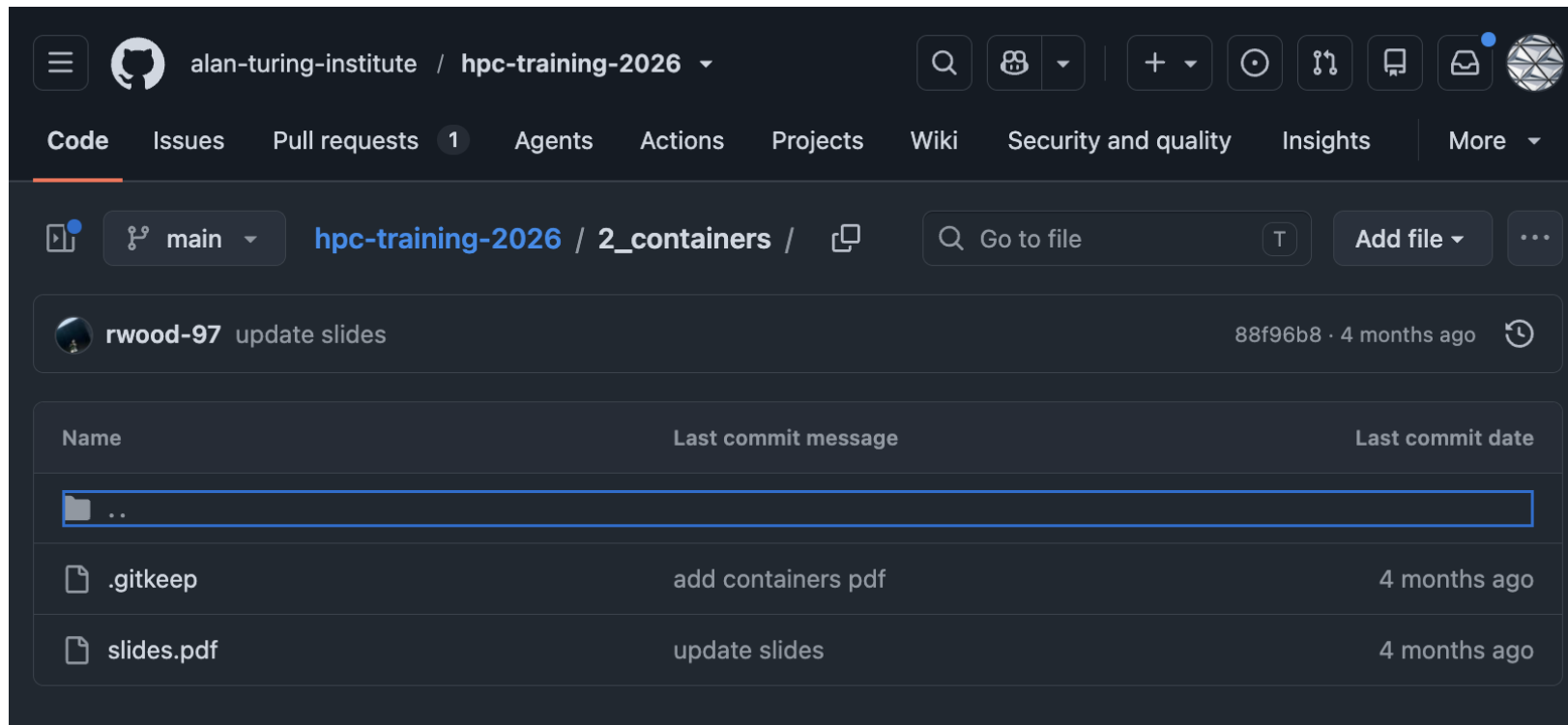
Containers (15:00-16:00)

Iain Stenson & Rosie Wood



Slides available at

- <https://tinyurl.com/is-ai26> @ 2_containers/slides.pdf



What are containers and why use them?

- Containers are a technology that allow applications to be packaged and isolated with their entire runtime environment.
- They can give the appearance of root access, on a system that you may not actually have root access to.
- They provide isolation from other (potentially conflicting) software installations.
- They may aid reproducibility.
- They can simplify development as well as deployment of software.
- Isambard-AI doesn't come with a lot of pre-installed software and containers are one way of adding your own.

Docker vs podman vs apptainer

- **podman-hpc** and **apptainer (a.k.a. singularity)** available on Isambard-AI
- Why not docker?
 - Security and open-source considerations
 - Docker requires root permissions on the host system by default vs. apptainer and podman-hpc can be run as user processes
- OCI and compatibility
- Why apptainer?
 - Works easily across multi-node configurations
 - Host MPI integration
 - Apptainer images are saved as `.sif` files

Container registries

- **Docker hub** (<https://hub.docker.com/>)
 - Widest range of available images.
 - You can create an account and push your own images.
- **Nvidia NGC containers** (<https://catalog.ngc.nvidia.com/?tab=container>)
 - PyTorch – has Python, PyTorch & CUDA pre-installed (see [here](#) for docs)
 - CUDA – has CUDA installed but you will need to install Python packages (see [here](#) for docs)
 -  ***Isambard-AI is AARCH64/ARM64 and those platforms should be preferred over AMD64.***
- **GitHub container registry** (docs [here](#))
 - To pull container images built with GitHub actions
 - e.g. <https://github.com/alan-turing-institute/reginald/pkgs/container/reginald>

Pulling an image

We've already pulled some images, You won't need to pull any new images for this session.

1. Check your system CUDA version:

- `srun --gpus=1 --time=1:15:00 --reservation=Turing_Workshop --pty /bin/bash`
- `nvidia-smi --version`
- You can use any image with:
 - container CUDA version $\leq$ your system CUDA version
 - the CUDA Forward Compatibility Package installed

2. Download the image you'd like to run:

- e.g. `apptainer pull docker://nvcr.io/nvidia/pytorch:24.12-py3`
- e.g. `apptainer pull docker://nvidia/cuda:12.6.3-cudnn-devel-ubuntu20.04`
- e.g. `apptainer pull docker://alpine` (you will need to install your own CUDA)

3. By default, this will save the image in the current working directory as a SIF file.

Run commands interactively in a container

1. If you haven't already, start an interactive job:
 - `$ srun --gpus=1 --time=1:15:00 --reservation=Turing_Workshop --pty /bin/bash`
2. Change directory:
 - `$ mkdir $PROJECTDIR/$USER && cd $PROJECTDIR/$USER`
 - `$ cd -P .` (moves you to the real path, avoiding symlinks)
3. Start an interactive shell in your container:
 - `$ apptainer shell --nv $PROJECTDIR/pytorch_24.12-py3.sif`
 - We have pulled this image already and put it in the project folder
 - `--nv` tells apptainer to use NVIDIA GPUs
4. Run commands in the container, e.g. check you have torch installed with CUDA:
 - `Apptainer> python -c "import torch; print(torch.cuda.is_available())"`
 -  In current set up, the container image is read-only and any python packages you install will be installed to ~/.local (i.e. your changes are local only)

Ways of running commands in a container

1. Start an interactive shell in the container:

- `apptainer shell [shell options...] <container.sif>`

2. Run a specific command in the container:

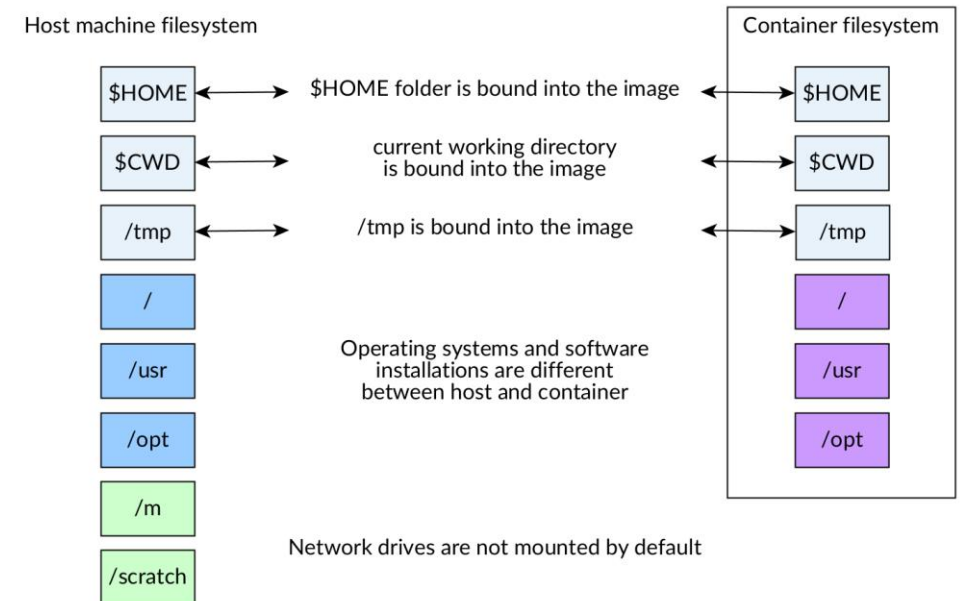
- `apptainer exec [exec options...] <container.sif> <command>`
- Best for batch scripts
- `apptainer exec --nv pytorch_24.12-py3.sif python -c "import torch; print(torch.cuda.is_available())"` does the same as we did earlier but in a single command

3. Run a pre-defined default command within the container:

- `apptainer run [run options...] <container.sif> [args...]`
- Default command is defined when the image is built
- Use `apptainer inspect -r` to see what it is.

Container filesystems

- By default, apptainer mounts several host directories into the container filesystem.
 - user's home directory (\$HOME)
 - current working directory (\$PWD)
 - /tmp, /dev, /proc, /sys and some others
 - These mounted directories are writable
- We can tell apptainer not to mount with e.g. `--no-mount tmp,dev`
- Or we can tell apptainer to mount additional directories with `--bind src[:dst[:opts]][,...]`
 - e.g. `apptainer shell --bind /scratch --nv myimage.sif`
 - ⚠ You may need to mount user-specific tmp on Isambard-AI with `--bind $TMPDIR:/tmp`
- Docker and podman have different behaviour (no default mounts).



Installing packages in your container

1. By default, the container filesystem is read-only (except for the mounted directories), i.e. the following will error:
 - `$ apptainer shell $PROJECTDIR/alpine_latest.sif`
 - `Apptainer> cd / && touch x`
 - As a consequence, we can't install packages in our containers with, for example, `apt install`.

Three options for installing packages:

1. Use a venv located in a mounted directory (e.g. `$CWD`)
 - Avoid using your `$HOME` directory due to issues with storage quota
 - Use the container python to create your venv, then pip install into it
2. Use a sandbox container
3. Write your own def file

Writeable filesystems /sandbox containers

You can interactively build your own container image using the `--sandbox` flag:

1. First, build a “sandbox” image from a base image:
 - `$ aptainer build --sandbox $TMPDIR/alpine $PROJECTDIR/alpine_latest.sif`
2. Edit it by, for example, installing software:
 - `$ aptainer shell --writable $TMPDIR/alpine`
 - `Apptainer> apk add python3 py3-pip`
 - `Apptainer> python3 --version`
3. And then turn the sandbox into .sif format with another build step:
 - `$ aptainer build alpine_python.sif $TMPDIR/alpine`

Building a custom apptainer image with a .def file

container.def

1. Choose a base image:
 - NVIDIA CUDA and PyTorch images are solid choices for most Isambard-AI use cases
 - Basic images (ubuntu, alpine, etc) - you then install everything you need
2. Write your .def file
3. Build your image with:
 - `apptainer build container.sif container.def`
 -  ***This should be done from a compute node***
4. Run your container image as before

```
Bootstrap: localimage
From: /path/to/pytorch_24.12-py3.sif

%post
# install python packages
pip install --no-cache-dir \
transformers==4.48.3 \
huggingface_hub[cli] \
datasets \
accelerate \
tokenizers

%environment
# set cache directory
export HF_HOME=/tmp/huggingface

%runscript
exec python "$@"
```

Multi-node training with apptainer

sbatch_pytorch_container.sh

- Same training code as for earlier lesson with BriCS team
- Isambard-AI requires multi-node specific setup steps:
 1. Add `module load brics/apptainer-multi-node` to your batch script
 2. Add `/host/adapt.sh` to the beginning of scripts/commands you will run in the container
- Isambard-AI docs [here](#)

```
#!/bin/bash
#SBATCH --job-name=Torch_Distributed
#SBATCH --nodes=2
#SBATCH --gpus=8
#SBATCH --time=00:10:00
#SBATCH --ntasks-per-node=4

module load brics/apptainer-multi-node

export MASTER_ADDR=$(scontrol show hostname $SLURM_NODELIST
| head -n 1) # e.g. nid001038
export MASTER_PORT=29600

echo "Job Started at $(date)"

# Run the function with srun
srun -N 2 \
    --gpus=8 \
    --mpi=pmi2 \
    --ntasks-per-node=4 \
    bash -c 'export WORLD_SIZE=$SLURM_GPUS; export
RANK=$PMI_RANK; export LOCAL_RANK=$SLURM_LOCALID; apptainer
exec --nv container.sif bash -c "/host/adapt.sh; python3
train.py"'

echo "Job Finished at $(date)"
```

Exercise: Build a container that behaves like a CLI tool

1. Find a useful tool, which isn't already available on Isambard-AI.
 - We suggest [ripgrep](#) (a faster grep), [azcopy](#) (for transferring data from Azure) or your coding agent of choice
2. Choose a base image.
 - Ubuntu and Alpine both have ripgrep available as packages.
 - `node:xx-alpine` is a good base image for claude-code
3. Create a sandbox image, so that you can verify the install commands.
 - i.e. `apt-get install <package>` or `apk add <package>` or follow from [docs/docs](#)
4. Write a definition file.
5. Build the image with the additional package(s) installed.
6. Make it executable with `chmod +x container.sif`
 - Optionally, make a symlink to it called "rg" with `ln -s /path/to/ripgrep.sif rg` . (or equivalent for azcopy or coding agent)
7. Add the enclosing directory to your PATH environment variable.
8. Use it as you would any other command:
 - `rg -e "pattern" -r /path/to/some/directory`
 - `azcopy copy source dest`

Exercise: Installing python packages using a venv

Repeat what we've just done using the venv approach:

1. Launch a container with the PyTorch image
 1. `cd -P .`
 2. `apptainer shell $PROJECTDIR/pytorch_24.12-py3.sif`
2. Create a venv and give it access to the system-wide packages
 1. `Apptainer> python -m venv --system-site-packages venv`
3. Activate your venv
 1. `Apptainer> source venv/bin/activate`
4. Install the packages needed for running the train script
 1. `Apptainer> pip install ...`
5. Adapt the batch script to use your venv, instead of the system packages
6. What are the pros and cons of using a venv vs installing dependencies in the build step?
 1. List two advantages
 2. List two disadvantages